

PRIVACY INFRASTRUCTURE

Hushmark Whitepaper

Private payments on Solana



Overview

Hushmark is a privacy-focused payment interface on Solana. It brings wallet connection, encrypted-note balances, zero-knowledge proof generation, transaction review, and settlement checks into one experience for SOL and USDC. The current implementation integrates Privacy Cash SDK 1.2.2 and its existing protocol. [1, 2]

The product addresses a specific problem: public transaction histories can reveal financial relationships and activity beyond an individual payment. Hushmark uses a pool-based protocol that aims to obscure the direct relationship between a deposit and a later withdrawal. The deposit and withdrawal addresses and amounts remain public; timing, address reuse, and service metadata can still reveal connections.

This paper explains the implementation, its trust assumptions, privacy boundaries, and development priorities. It is intended for users, collaborators, and developers evaluating how Hushmark works. On 24 September 2026, the project owner reported successful tests using real SOL and USDC. That functional result is recorded separately from the automated checks and the planned independent security review.

Reading guide

- 2 Purpose and product scope
- 3 System architecture
- 4 Payment lifecycle
- 5 Privacy model
- 6 Key handling and operational safeguards
- 7 Validation and development priorities
- 8 References and terminology

Purpose and product scope

The problem

Public ledgers make transactions inspectable. When a wallet is used repeatedly, its activity can also expose patterns: where funds arrive from, which addresses receive them, and when balances change. A single payment may therefore reveal information beyond the amount and recipient involved in that payment.

Hushmark focuses on making an existing privacy protocol accessible through a coherent payment flow. Its contribution is the application layer: local key handling, note discovery, clear payment review, transaction checks, and an interface that explains what is visible. The underlying pool, circuits, and relaying infrastructure come from Privacy Cash. [1, 2]

Current scope

The implementation targets Solana mainnet and supports SOL and USDC. Users connect a compatible Phantom or Solflare software wallet, unlock a private-note account, deposit supported assets, scan encrypted notes, and request a withdrawal to a separate public wallet. The browser generates the protocol proof using locally bundled proving assets.

The current product is a single-chain interface. It does not introduce a separate blockchain, consensus mechanism, or original cryptographic protocol. This paper describes the implemented integration rather than an assertion that every form of onchain activity becomes private.

Cryptographic building blocks

The integrated SDK uses Groth16 proofs with inputs such as note commitments, Merkle paths, and nullifiers. New V2 notes use AES-256-GCM encryption, while the SDK retains legacy decryption support. Hushmark consumes these upstream constructions and their proving assets; their security is a protocol dependency. [1, 2]

Design priorities

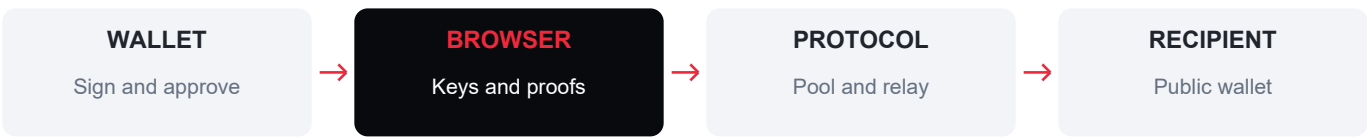
- Clear consent: show the asset, amount, destination, and relevant fees before an operation is confirmed.
- Recoverable access: check that the wallet reproduces the signature used to derive private-note keys.
- Honest balances: display an unread state until notes have been scanned and decrypted.
- Verifiable outcomes: distinguish submission from finalization and reject mismatched transaction responses.
- Explicit privacy boundaries: explain remaining public information and service dependencies.

These priorities guide both interface decisions and the development roadmap. They are implemented as application checks, supported by the protocol's own transaction and proof requirements. [1]

System architecture

Components and responsibilities

Hushmark is delivered as a static web application. Its browser bundle integrates the Privacy Cash SDK, while a connected wallet supplies the signatures needed for account access and deposit approval. Proof generation runs in the browser. Solana RPC and Privacy Cash services provide network data and relay operations. [1-4]



Conceptual flow. The browser also reads Solana RPC and protocol note data.

Component	Responsibility
Connected wallet	Supplies the account public key, signs the protocol unlock message, and approves deposit transactions.
Hushmark browser application	Derives note keys, scans and decrypts notes, generates proofs, presents a review, and checks outgoing and returned transaction data.
Privacy Cash protocol	Supplies the pool program, note and proof construction, proving assets, and relayer integration used by the SDK.
Solana RPC and network	Provide account and transaction reads, execution results, and the finality information shown in the interface.

Integration boundary

The application bundles the SDK and the matching transaction circuit files. This keeps the deployed interface and its proving assets together. The selected RPC is checked against the expected Solana mainnet genesis hash, and the configured protocol program must be executable. Those checks identify the intended network and program; they do not constitute a review of the program's security. [1]

A public recipient receives a withdrawal through the protocol flow. Hushmark does not collect a wallet seed phrase. Its interface does have temporary access to derived private-note keys while the session is unlocked, so the delivered application code and the browser are part of the trust boundary.

Payment lifecycle

1 Connect and unlock

The user selects a supported software wallet. Hushmark requests the exact protocol message "Privacy Money account sign in" twice, verifies both signatures, and checks that they match. The signature is used to derive private-note keys. A local fingerprint helps detect a change in the recovery signature on a later visit. A mismatch blocks unlocking. [1, 3]

2 Review and deposit

The user chooses SOL or USDC and an amount. The interface checks the public balance, required SOL reserve, network, and fee configuration. After review, the SDK generates a proof and constructs a deposit transaction. Hushmark checks the transaction against the approved operation before requesting the wallet signature and allowing submission.

The public wallet funds the pool. This funding transaction remains visible on Solana. The protocol creates encrypted note data that the user can later use to reconstruct the private balance.

3 Synchronize the private balance

The browser fetches encrypted notes, decrypts relevant records, and checks whether they remain unspent. The private balance is reconstructed from available notes rather than increased optimistically after a button click. A first synchronization can take time because it may need to inspect substantial historical data. [1, 4]

4 Review and withdraw

The user chooses a separate recipient address and the amount to debit from the private balance. The review presents the withdrawal fee and expected net amount received. The current integration checks whether two available notes can cover the requested amount and blocks a partial send when they cannot.

After confirmation, the browser generates the withdrawal proof and submits the reviewed operation to the relay. The payload is checked against the approved recipient, asset, amount, and fee. A withdrawal can proceed using the unlocked note keys without another wallet popup; the interface's confirmation is therefore a meaningful authorization. [1, 5]

5 Check the network result

A returned signature starts a settlement check. The interface distinguishes submitted, missing, failed, and finalized outcomes. For withdrawals, it also checks that the returned transaction contains the approved proof. When the result is uncertain, the transaction and balance need to be checked before retrying.

Privacy model

Protection objective

The integrated protocol aims to protect the direct link between a deposit and a withdrawal. Zero-knowledge proofs establish the validity of a private-note operation under the protocol's rules. They do not remove the surrounding public transactions or prevent every form of observation. Privacy depends on both the protocol and the way it is used. [1, 2]

Information	Visibility or treatment
Deposit funding address and amount	Public on Solana.
Withdrawal recipient and amount	Public on Solana.
Direct deposit-to-withdrawal relationship	The protocol aims to obscure this relationship through its pool and proof system.
Decrypted notes and derived spending keys	Handled in browser memory during an unlocked session.
Cached note data and scan metadata	Stored for the tab and scoped to the connected account; metadata is not all encrypted.
Requests to RPC and relayer services	Visible to the contacted providers, including relevant request and connection metadata.

Remaining ways to infer relationships

An observer may correlate similar amounts, short time intervals, repeated destinations, or a small set of plausible transactions. Reusing the funding address as a withdrawal recipient directly weakens the intended separation; the current interface rejects that recipient choice. The size and activity of the relevant pool also affect the set of possible relationships an observer can consider.

Relayers and RPC providers remain additional observers. In the integrated SDK flow, requests for indices associated with matching encrypted notes reveal those queried identifiers to the relayer. Hosting and network infrastructure may also receive ordinary connection metadata. This paper makes no promise of protection against a compromised browser, a malicious application build, or comprehensive traffic correlation.

Practical interpretation

An encrypted private balance and a valid proof are useful components of privacy. Their protection must be evaluated alongside the public transaction endpoints, service access, and operational behavior described here. Hushmark documents these boundaries so users can make informed decisions about the protection available.

Key handling and operational safeguards

The unlock signature is sensitive

The protocol-message signature is an input to key derivation. Anyone who obtains the same signature may be able to derive the associated private-note keys. A request to sign this message must therefore be treated as access to the private-note account, even when the wallet presents it as message signing. [1, 3]

Hushmark checks two matching signatures, stores a recovery fingerprint locally, and derives keys in the browser. Losing access to the original wallet or to its ability to reproduce the required signature can prevent recovery. The interface requires the user to confirm they are using a software wallet; hardware wallets are outside the supported recovery assumptions.

Session and storage boundaries

Encrypted note data, scan offsets, and matching-note metadata are cached in tab session storage under an account-specific prefix. A recovery fingerprint is stored in local storage. The application does not intentionally persist decrypted notes or derived keys as a saved account database.

Locking the session releases the active key objects, clears in-memory application state, and invalidates outstanding approvals. Browser memory management does not provide a guarantee of complete forensic erasure. A compromised device or injected application script remains outside the protection offered by these interface controls.

Checks around user approval

The integration compares outgoing relay requests with the operation the user reviewed. Deposit checks include the transaction message, intended program, amount, fees, and signature. Withdrawal checks cover recipient, asset, amount, fee, and returned proof data. Changes to the form, account, provider, or session invalidate an approval rather than allowing stale details to proceed. [1]

Network and deployment dependencies

Fees and minimum amounts can change and are read from protocol configuration. The interface checks relevant changes before submitting. RPC and relay calls have timeouts, and an interrupted response can still require a network check to determine whether a transaction was submitted.

Deployment must preserve the application bundle and matching proving assets, use HTTPS, and allow required protocol and RPC requests. Availability depends on those services and on the user's device. An independent security review of Hushmark remains a development priority.

Validation and development priorities

Evidence to date

On 24 September 2026, the project owner reported personally testing Hushmark with real SOL and USDC and obtaining successful results. This paper records that result as owner-reported functional testing. The report is separate from the automated checks below and from independent security assurance. [1]

Evidence	Scope
Owner testing with real assets	Successful SOL and USDC functional testing reported by the project owner on 24 September 2026.
Automated implementation checks	Amount precision, fees, account isolation, approval changes, real proof generation, synthetic-note recovery, and mismatched response rejection were exercised.
Independent security review	Planned; no completed independent audit of Hushmark is claimed.

The automated payment checks used actual cryptographic proof generation and transaction assembly, with fixture wallets and synthetic notes. Relay submissions and settlement responses were intercepted. Their finalization labels represent test fixtures rather than independently observed transfers. Detailed test scope and repeat commands are included in the implementation record supplied with this package.

Development priorities

The current integration provides the core private-payment flow. The following priorities guide further work; they are not dated delivery commitments.

- Reproducible verification: document end-to-end scenarios, recovery after reload, and the supported wallet and device matrix while protecting test-wallet privacy.
- Independent review: assess signature-derived keys, session handling, transaction checks, cryptographic dependencies, and deployment integrity.
- Reliability: improve behavior during lengthy note scans, service interruptions, fee changes, and uncertain transaction results.
- Usability and documentation: make recovery requirements, payment authorization, and privacy boundaries easier to understand and inspect.

Progress should be recorded as concrete implementation changes and test results. A successful functional test demonstrates the tested behavior; broader assurance requires review across the relevant failure and threat scenarios.

References and terminology

Implementation references

- [1] Hushmark implementation snapshot, 25 September 2026. The accompanying website-source folder includes README.md, VERIFICATION.md, package.json, build-privacy.mjs, and src/privacy.js, src/privacy-sdk.js, src/privacy-guards.js, and src/config.js. These files establish the application-specific behavior described in this paper.
- [2] Privacy Cash SDK. Hushmark pins dependency version 1.2.2. Upstream repository and documentation may evolve beyond that version. <https://github.com/Privacy-Cash/privacy-cash-sdk>
- [3] Privacy Cash frontend integration and recovery guidance. <https://privacypash.mintlify.app/sdk/frontend>
- [4] Privacy Cash balance and note-reading integration. <https://privacypash.mintlify.app/sdk/balance-fe>
- [5] Privacy Cash deposit and withdrawal integrations. <https://privacypash.mintlify.app/sdk/deposit-fe>
<https://privacypash.mintlify.app/sdk/withdraw-fe>

Terminology

Encrypted note: protocol data representing an amount and the information needed to identify and spend it with the corresponding keys.

Zero-knowledge proof: a cryptographic proof that establishes a specified statement while withholding the private witness used to produce it. Its guarantees apply to the statement and protocol construction involved.

Relayer: a service used by the protocol integration to submit or coordinate operations and provide related protocol data. It is also a point of availability and metadata dependence.

Unlinkability: the intended difficulty of determining which deposit is related to a withdrawal. It depends on the protocol and surrounding observable behavior.

Finality: the network confirmation state used by the interface to distinguish completed settlement from submission or an unresolved result.

Document scope

This version describes the Hushmark integration and the evidence available as of its publication date. It should be updated when implementation, protocol dependencies, or testing status change. The inclusion of upstream references identifies dependencies and does not imply affiliation or endorsement.